

PeakBaleen: Peak-Aware Flash-Cache Admission via Ising-Machine Episode Selection

An improvement on Baleen: ML Admission & Prefetching for Flash Caches (USENIX FAST '24)

Prepared by

Ching-Hao Chiu · Jie Fu · Sixue Xing
University of Notre Dame, Notre Dame

1. Main Functions and Expected Significance

Bulk storage systems — blobstores, data warehouses and the file systems beneath them — are still built on hard disks, because HDDs remain the cost-effective medium for capacity. They are not, however, cost-effective for throughput: a disk delivers roughly 100 IOPS regardless of how large it becomes. Data-center operators therefore place a flash cache in front of the disks, and the number of backend disks they must buy is set by how much load that cache fails to absorb at the busiest moment of the day.

Flash, in turn, wears out as it is written, so a production flash cache runs under a long-term average write-rate budget (for example, three drive-writes per day). Every flash write is scarce, and the admission policy must decide which of many candidate blocks deserves one. Baleen (FAST '24) is the state of the art for that decision: it introduces an offline residency model called episodes, trains gradient-boosted models to imitate an episode-based optimum, and — crucially — optimizes Disk-head Time (DT) rather than hit rate. Baleen reduces Peak DT by 12% over the best prior policy at a fixed flash write rate, and by 17% in estimated total cost of ownership when it is also allowed to choose the write rate [1].

The function this project adds is peak-awareness of the spending decision itself. Provisioning follows the peak: Peak DT, defined as the P100 of backend utilization measured over 10-minute windows, is proportional to the number of backend servers required [1, §3.1]. Baleen's selection rule, however, ranks episodes by DT saved per byte written and admits them in that order until the write budget is exhausted. That rule maximizes total — that is, average — DT saved; nothing in it refers to when the saving lands. The paper is explicit that this was a deliberate deferral: “As explicitly optimizing admission for the peak introduces significant complexity, we leave that for future work” [1, §3.1].

We propose to close that gap by reformulating episode selection as a joint min–max problem across time windows — spend the same flash writes so as to minimize the worst window rather than the total — expressing it as a Quadratic Unconstrained Binary Optimization (QUBO) problem, and solving it with an Ising machine. The resulting selection is then distilled into an online machine-learning policy exactly as Baleen distills its offline optimum, so the cache datapath is unchanged and no solver runs at request time.

Expected significance. Peak DT translates one-for-one into backend hardware under the paper's own cost model, so a further 5% reduction at an unchanged write rate is a 5% reduction in the disks a cluster must provision. Beyond the number, the project produces three reusable results: (i) the first Ising/QUBO

formulation of budget-constrained flash-cache admission that we are aware of; (ii) a quantitative answer to a question the Baleen authors raised and left open — how much of the remaining 16% gap between Baleen and its own offline optimum [1, §5.6] is attributable to peak-blindness; and (iii) an attribution analysis of why peak optimization is hard, which is a publishable finding whether the headline number improves or not.

2. Rationale and Related Background

2.1 How Baleen decides what to cache

Baleen models the backend cost of a miss with Disk-head Time: for one IO fetching n bytes, $DT = t_{\text{seek}} + n \cdot t_{\text{read}}$, so DT is a weighted sum of IO misses and byte misses and captures both the IOPS and the bandwidth limits of a disk. Backend load in a time window is the DT needed to serve misses normalized by provisioned DT, and Peak DT is the maximum of that quantity over 10-minute windows [1, §3.1]. The paper validates the formula against production disk utilization at Meta, where the peaks line up within 1%.

To decide admissions, Baleen groups the accesses a block would receive during one cache residency into an episode, and scores each episode by the DT it would save divided by the flash writes it would cost. Episodes are then admitted in descending score order until the write-rate budget is exhausted; this offline procedure (called OPT) provides the labels that the online gradient-boosted admission and prefetching models are trained to imitate [1, §3.4–3.6]. Prefetching is coordinated with admission through the same episode model, which determines both when to prefetch (ML-When) and what range to prefetch (ML-Range).

2.2 Why the policy is not peak-aware, and what the authors already tried

The ranking rule is a greedy solution to a knapsack over a single, time-agnostic objective. Two episodes with identical DT-saved-per-byte are interchangeable to it even when one saves its DT in the busiest ten minutes of the day and the other saves it at 4 a.m. Since the peak sets provisioning, those two are worth very different amounts.

The Baleen authors recognized this and made one attempt at it: they extended the policy so that it admitted only during periods of high load. Their report of the outcome is unambiguous: they “found that while this saved flash writes, it did not reduce Peak DT”, and concluded that “more fundamental changes (e.g., scoring episodes by their usefulness in reducing Peak DT) will be required to optimize explicitly for peak load” [1, §5.6]. The appendix explains the underlying difficulty: the peak window is itself policy-dependent, because “one policy may be good at dealing with a traffic pattern that causes peaks for other policies, but be foiled by a pattern that is handled well by others. This makes optimizing the peak a whack-the-mole game” [1, App. A.3].

Evidence from the released artifact. We inspected the code that accompanies the paper. It already contains the skeleton of the failed attempt: `policies.py` defines `PolicyUtilityPeakServiceTimeSize` and `PolicyUtilityPeakServiceTimeWeightedSize`, whose scores divide a per-episode quantity `peak_service_time_saved` by the writes required, and `episodes.py` computes that quantity only after being given a single peak interval as an oracle parameter (`peak_ts1_start`, `peak_ts1_end`; the constructor asserts that both are supplied). The shipped peak policy is therefore (i) scored per episode and still greedy, (ii) restricted to one window, and (iii) dependent on knowing where the peak is in advance. This is valuable to us in two ways: it is a ready-made baseline that we can run rather than reimplement, and it identifies precisely which three properties our design must change.

The missing ingredient is that the decisions are coupled. Episodes compete for one shared write budget, and their savings land in different windows; removing load from the tallest window can promote a different window to be the new maximum. A per-episode score cannot represent that coupling, whereas a joint objective over all windows can. This is why we treat selection as a combinatorial optimization rather than as a better ranking function.

2.3 Why an Ising machine

Minimizing the maximum window load subject to a global write budget is a multi-window (min–max) knapsack problem: NP-hard, with a quadratic interaction structure once the objective is written as a penalty on window loads. That is exactly the class of problem Ising machines target. A QUBO is the canonical input format for them [5], and modern solvers — simulated annealing and, more recently, simulated bifurcation, which maps the problem onto a network of nonlinear oscillators and is highly parallel on GPUs [6, 7] — routinely handle 10^3 – 10^5 binary variables. Using one here is not incidental: the quadratic penalty on per-window load produces dense couplings between every pair of episodes that share a window, which is the structure these machines are built for, and which a greedy ranking cannot express.

It is worth stating where the solver does and does not run. Baleen already performs an expensive offline pass to produce labels; our solver replaces that pass. The online policy remains a gradient-boosted model evaluated per request, so no combinatorial solver is ever placed in the cache datapath (Figure 1).

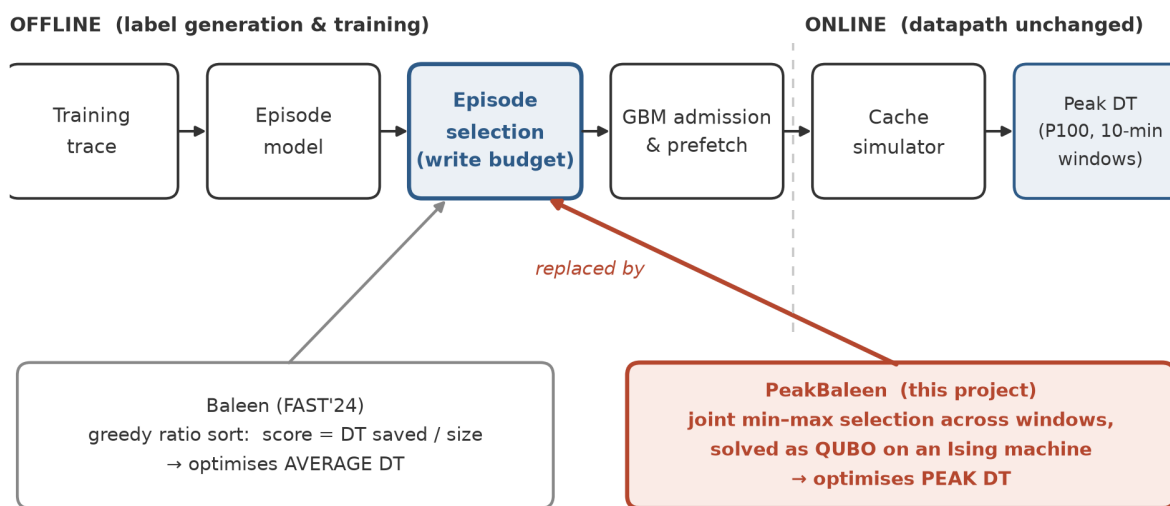


Figure 1. Where PeakBaleen changes Baleen. Only the offline episode-selection stage is replaced; the trained models and the online datapath are untouched.

3. Design Hypotheses and Challenges

3.1 Hypotheses

H1 (primary). Selecting episodes by a joint min–max criterion across 10-minute windows, under the same flash-write budget, reduces Peak DT by at least 5% relative to stock Baleen, averaged over the seven Meta traces. We test H1 by running both policies through the unmodified simulator, which already reports Peak DT, at the paper's operating point (400 GB cache, 3 drive-writes per day).

H2 (method). A quadratic (sum-of-squares) penalty on window loads, refined by iterative reweighting, is a sufficient surrogate for the true min–max objective, and an Ising solver on that surrogate beats both (a) the greedy peak-scoring policy already present in the artifact and (b) a greedy warm start, by at least 3% Peak DT. We test H2 by holding the formulation fixed and varying only the solver, including an exact integer-programming solution on down-sampled instances to measure the optimality gap.

H3 (deployability). An online model trained on peak-aware labels, using only causal load features (trailing backend load, time-of-day) and no oracle peak window, retains at least 60% of the offline gain. We test H3 by comparing three configurations: offline selection with oracle windows, the trained online policy, and stock Baleen.

3.2 Challenges

Peak migration (“whack-a-mole”). Suppressing the tallest window can simply promote the second-tallest, which is what defeated the authors' first attempt. Our objective is defined over all windows simultaneously precisely for this reason, but it means the benefit is bounded by the flatness of the load profile: if the profile is already flat, there is little to win. We will measure the profile's peak-to-mean ratio per trace first (the paper reports about 2) and use it to predict the achievable headroom before spending effort on the solver.

Problem size. A trace-day yields on the order of 10^5 – 10^6 episodes, which is far beyond a dense QUBO. We will exploit the fact that episodes are time-local — an episode's residency is bounded by the eviction age — so windows couple only episodes that overlap them. The trace is partitioned into overlapping time blocks, and within a block we keep only episodes that touch the most loaded windows, which is where the objective actually has gradient. Measuring the resulting instance sizes is an explicit Phase-1 deliverable, not an assumption.

Surrogate fidelity. Sum-of-squares is not the maximum. Iterative reweighting drives the surrogate toward min–max, but convergence is not guaranteed; we will report the true Peak DT of every intermediate solution so that the surrogate is never the thing being evaluated.

Keeping the write budget honest. A peak-aware selection is only meaningful at an unchanged write rate. The QUBO enforces the budget with a penalty term, which is soft; we therefore repair any overshoot after solving and re-run Baleen's existing eviction-age and threshold convergence loop so that the simulated write rate lands on target. Every reported comparison must be at matched write rate, and we will report the achieved rate alongside Peak DT.

Offline-to-online transfer. The offline selection uses knowledge of the whole trace, including where the peaks are. The online policy cannot. There is a real risk that the gain is an artifact of hindsight, and a second risk of overfitting to one day's peak. We will train on day 1 and test on the remaining days, as the paper does, and treat the oracle configuration as an upper bound rather than a result.

Compute budget. An ML simulation run takes roughly 30 minutes, and the evaluation matrix (7 traces $\times$ 3 samples $\times$ several policy variants) is large. The work is CPU-parallel and fits our 16-core, 125 GiB machine, with the solver optionally on the GPU, but the schedule must respect it; we size each phase accordingly in Section 5.

3.3 Most critical challenge

The critical risk is not the solver — it is whether an offline peak-optimal selection survives the move to an online, causal policy at all. The authors' negative result shows that a naive load-triggered policy does not reduce Peak DT, and their whack-a-mole observation explains why: the peak an online policy would have to anticipate depends on what that policy itself does. If the gap between our oracle configuration and our online configuration turns out to be most of the gain, the honest conclusion is that peak-aware admission needs prediction of future load, not merely a better offline objective.

We treat that outcome as a result rather than a failure, and we design the evaluation to produce it cleanly: the three-configuration comparison in H3 separates “the objective is wrong” from “the information is unavailable online”, and the per-window attribution analysis identifies where the peak reduction is lost — for example to late admissions, which the paper measures at 9% of DT [1, §5.6], or to windows in which prefetching does not pay, which the appendix identifies as Baleen's worst intervals [1, App. A.3].

4. Approach and Plan

4.1 Core idea: selection as a min–max problem, solved as a QUBO

Let each candidate episode e carry a binary decision $x_e \in \{0, 1\}$ (admit or not), a flash-write cost s_e , and a per-window saving $d_{e,w}$: the DT it would remove from window w if admitted. Let C_w be the backend load of window w with no admissions, and W the flash-write budget. The load of window w under a decision vector x is:

$$L_w(x) = C_w - \sum_e d_{e,w} \cdot x_e$$

Baleen's offline optimum maximizes the total saving subject to the budget, which is a knapsack solved greedily by the score DT saved / size:

$$\max_x \sum_e \sum_w d_{e,w} \cdot x_e \quad \text{s.t.} \quad \sum_e s_e \cdot x_e \leq W$$

Our formulation keeps the same feasible set but changes the objective to the quantity that actually sets provisioning — the worst window:

$$\min_x \max_w L_w(x) \quad \text{s.t.} \quad \sum_e s_e \cdot x_e \leq W, \quad x \in \{0, 1\}^n$$

A maximum is not quadratic, so we optimize a weighted sum-of-squares surrogate with a penalty for the budget. With window weights α_w and penalty strength λ :

$$\min_x \sum_w \alpha_w \cdot L_w(x)^2 + \lambda \cdot (\sum_e s_e \cdot x_e - W)^2$$

Expanding the squares gives exactly a QUBO — the quadratic coefficient coupling two episodes is $\sum_w \alpha_w d_{e,w} d_{e',w} + \lambda s_e s_{e'}$, so episodes interact whenever they save DT in the same window or compete for the same budget. Raising the exponent on the load term would approach the true maximum; instead of doing so directly, we keep the problem quadratic and re-weight: after each solve we set α_w in proportion to a power of the current load L_w , so that windows which remain tall attract more weight in the next round. Substituting $x_e = (1 + \sigma_e)/2$ converts the QUBO to an Ising Hamiltonian over spins $\sigma_e \in \{-1, +1\}$, which is the native input of the solvers.

Solvers. We will compare three, all installable without root on our machine (verified): simulated bifurcation (GPU-accelerated, PyTorch-based) as the primary Ising machine; simulated annealing as a CPU reference; and an exact integer-programming solve on down-sampled instances to measure how far from optimal the Ising

solutions are. A greedy warm start (Baleen's own ranking) seeds the solvers, so our result can never be worse than the baseline by more than solver noise.

From a solution to a policy. The selected set is converted to training labels in the format Baleen already uses, and the gradient-boosted admission model is retrained on them, with causal load features added so that the online policy can modulate its threshold with observed load. The prefetch gate (ML-When) is given the same feature, tightening off-peak and loosening near peak. Nothing in the simulator's datapath changes, which keeps the comparison honest: the only difference between the baseline and our system is the label-generating objective.

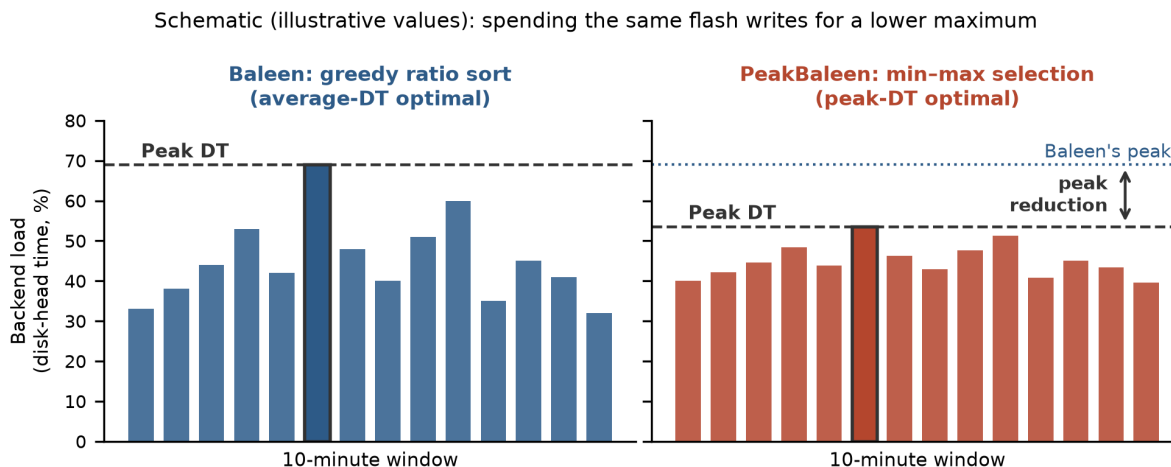


Figure 2. The intended effect, drawn schematically. Both panels spend the same flash writes. Greedy selection minimizes the average window load; min-max selection flattens the profile, accepting slightly higher load in quiet windows to lower the maximum, which is what sets the number of backend disks.

4.2 Baseline reproduction and experimental setup

Phase 1 reproduces Baleen before changing anything. The artifact carries USENIX “Available, Functional and Reproduced” badges; the simulator is pure Python, the seven Meta traces used for the headline figures are public, and the sample-0 trace set is 17 MB, so the first milestone is cheap. We will reproduce the per-trace peak backend load of the paper's Figure 9 (reported averages: 28.11% for Baleen against 31.95% for RejectX) and treat a match within run-to-run variance as the exit criterion for Phase 1. We will also build the per-window DT extractor that both the objective and the evaluation need, since the released harness reports only the aggregate Peak DT.

All experiments run on one workstation: 16 cores / 32 threads, 125 GiB RAM, one 24 GB GPU, no root access. The cache configuration follows the paper (400 GB flash, 3 drive-writes per day). Every comparison is run at matched flash write rate, and we report the achieved rate with each result.

4.3 Where the code changes

- `episodes.py` — attribute each episode's DT saving to the 10-minute windows its accesses fall in, producing the de,w matrix, and remove the single-oracle-window restriction of the existing peak code.
- `policies.py` — add a selection policy that builds the QUBO, calls a solver, applies the budget repair, and returns the admitted set, alongside the existing greedy policies so both can be run from one harness.

- `qubo/` (new module) — instance construction, time-block decomposition and candidate pruning, the three solver back-ends, and the reweighting loop.
- `train.py` — feed the new labels through the existing eviction-age and threshold convergence loop so the write-rate constraint is still met.
- `prefetchers.py` — add the load-adaptive gate to ML-When.

4.4 Evaluation plan and success criteria

- Primary metric: Peak DT at a fixed flash write rate, per trace and averaged over the seven traces, against stock Baleen and against RejectX. Success for H1 is a $\geq 5\%$ mean reduction with no trace regressing by more than 2%.
- Secondary metrics: mean DT, IO and byte miss rate, achieved flash write rate (constraint satisfaction), and the paper's estimated-TCO function, so that a peak gain bought by spending more flash cannot be mistaken for a win.
- Ablations: (a) the greedy peak-scoring policy already in the artifact; (b) our formulation with oracle peak windows; (c) our online policy without oracles; (d) solver ablation — simulated bifurcation, simulated annealing, greedy warm start, and exact ILP on down-sampled instances; (e) window granularity (5 / 10 / 30 minutes).
- Reporting: per-trace deltas rather than only the mean, three trace samples per configuration, and the achieved write rate for every row. Peak DT is a maximum and therefore noisy, so we also report the P99 and P95 of window load as stability checks.
- Attribution: for the traces where the gain is small, a breakdown of the peak window's DT into late admissions, prefetch-ineffective load, and unavoidable compulsory misses.

4.5 Risks and fallbacks

- If pruned instances are still too large for the solver, we reduce the time-block span and the candidate set, and report the size/quality trade-off — a result in itself about the granularity at which peak-aware selection is worth solving.
- If the Ising solver does not beat a greedy warm start, the contribution becomes the formulation plus a solver-quality study, with the ILP gap quantifying how much is left on the table.
- If the online policy loses most of the offline gain, we report the oracle-versus-online decomposition as the project's main finding, which directly answers the question the Baleen authors left open.
- Dedicated Ising hardware is not required at any point: simulated bifurcation on our GPU and simulated annealing on CPU are the planned solvers.

4.6 How we will demo

The demo is a single script that takes one trace, runs stock Baleen and PeakBaleen end to end at the same write rate, and prints a side-by-side table of Peak DT, mean DT, miss rates and achieved write rate, followed by a plot of backend load per 10-minute window for both policies — the real version of Figure 2. A second, smaller script shows the optimization itself: instance size, solver wall-clock time, energy trajectory, and the resulting objective compared with the greedy warm start.

4.7 How to reproduce

Reproduction needs no privileged access and no special hardware. A reviewer creates a conda environment from our specification, clones the Baleen artifact and our patch, downloads the 17 MB public trace bundle with the script the artifact provides, and runs one driver script; the GPU is optional, since the annealing back-end runs on CPU. We will pin every dependency version, publish the exact commands, and include the per-window extractor so that Peak DT can be recomputed from raw simulator output rather than taken on trust.

4.8 Task breakdown

Ching-Hao Chiu — baseline reproduction and measurement infrastructure: build the artifact, match the paper's Figure 9 numbers, implement the per-window DT extractor and the evaluation harness, own the write-rate matching procedure, and run the final evaluation matrix.

Jie Fu — optimization: window attribution, QUBO construction, time-block decomposition and pruning, the three solver back-ends and the reweighting loop, the ILP optimality study, and the solver ablation.

Sixue Xing — policy and learning: integrate the solver output into the episode-selection stage, regenerate labels, retrain the admission and prefetch models with causal load features, own the offline-to-online comparison and the attribution analysis, and coordinate the write-up.

All three members share the writing of the final paper and the final presentation. Weekly integration is on the shared workstation, with results tracked in a single per-trace results table so that every claim in the paper is backed by a stored run.

5. Timeline and Milestones

Dates (2026)	Phase	Deliverable / exit criterion
Sep 10 – 20	Literature review	Venue survey; Baleen chosen; artifact audited (builds, data sized)
Sep 21 – Oct 5	Baseline & problem definition	Baleen reproduced on 7 traces; Fig. 9 peak-load numbers matched; per-window DT extractor; QUBO spec frozen
Oct 6 – 20	Algorithm design	Min–max surrogate + reweighting; QUBO builder; solver comparison (SB / SA / ILP) on pruned instances
Oct 21 – Nov 1	Implementation & analysis	PeakBaleen policy in BCacheSim; offline-oracle Peak DT measured vs Baleen; online GBM trained on peak-aware labels
Nov 2 – 15	Full evaluation	All 7 traces × 3 samples; ablations; per-trace results table; attribution analysis
Nov 16 – Dec 2	Write-up & final talk	Final paper (conference format) + presentation; artifact and scripts released

The schedule follows the plan presented in our proposal talk. Two checkpoints are hard gates: at the end of Phase 2 we must have reproduced the baseline, and at the end of Phase 4 we must have an offline result. If

the offline result is negative, Phase 5 shifts from tuning to attribution — measuring why, which is the outcome we would then report.

6. References

- [1] Wong, Daniel Lin-Kit, et al. "Baleen:{ML} admission & prefetching for flash caches." 22nd USENIX Conference on File and Storage Technologies (FAST 24). 2024.
- [2] Berg, Benjamin, et al. "The {CacheLib} caching engine: Design and experiences at scale." 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). 2020.
- [3] Pan, Satadru, et al. "Facebook's tectonic filesystem: Efficiency from exascale." 19th USENIX Conference on File and Storage Technologies (FAST 21). 2021.
- [4] Yang, Tzu-Wei, et al. "{CacheSack}: Admission optimization for google datacenter flash caches." 2022 USENIX Annual Technical Conference (USENIX ATC 22). 2022.
- [5] Lucas, Andrew. "Ising formulations of many NP problems." *Frontiers in physics* 2 (2014): 5.
- [6] Goto, Hayato, Kosuke Tatsumura, and Alexander R. Dixon. "Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems." *Science advances* 5.4 (2019): eaav2372.
- [7] Mohseni, Naeimeh, Peter L. McMahon, and Tim Byrnes. "Ising machines as hardware solvers of combinatorial optimization problems." *Nature Reviews Physics* 4.6 (2022): 363-379.